



**DEPARTMENT OF CAREER
PLANNING & DEVELOPMENT**
Discover. Learn. Empower.

**NAAC
GRADE A+**
ACCREDITED UNIVERSITY



**RANKED
#19**
AMONGST TOP
UNIVERSITIES IN INDIA



ADBMS-Expt 1

Name: Akshra Ahuja
Class: 24AIT-KRG-1-B
UID: 24BAI70523

Submitted To: Alok Sir



Experiment 01

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than \$a.
2. **"Average Salary"**: All monthly salaries in the inclusive range [\$20,000, \$50,000].
3. **"High Salary"**: All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

Output Table

category	accounts_count
Low Salary	1
Average Salary	1
High Salary	3

	category text	accounts_count bigint
1	Low Salary	1
2	Average Sala...	1
3	High Salary	3



Experiment 01

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Output Table:

name	dept_name	salary
Alice	HR	50000
Bob	HR	20000
Charlie	IT	80000
David	IT	40000
Emma	Sales	45000

Employee/postgres@PostgreSQL 18

No limit

Query Query History

```
10     dept_id INT REFERENCES Departments(dept_id)
11 );
12
13 CREATE TABLE Salaries (
14     emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
15     salary INT NOT NULL
16 );
17
18
19 INSERT INTO Departments (dept_id, dept_name) VALUES
20 (10, 'HR'), (20, 'IT'), (30, 'Sales');
21
22 INSERT INTO Employees (emp_id, name, dept_id) VALUES
23 (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
24
25 INSERT INTO Salaries (emp_id, salary) VALUES
26 (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
27 -- 1.
28 select * from Employees e
29 left join Salaries s on e.emp_id=s.emp_id
30 left join Departments d on e.dept_id=d.dept_id;
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 5

Page No: 1

of 1

	emp_id integer	name character varying (50)	dept_id integer	emp_id integer	salary integer	dept_id integer	dept_name character varying (50)
1	1	Alice	10	1	50000	10	HR
2	2	Bob	10	2	25000	10	HR
3	3	Charlie	20	3	80000	20	IT
4	4	David	20	4	40000	20	IT
5	5	Emma	30	5	45000	30	Sales

The screenshot shows the DBeaver interface connected to a PostgreSQL database named "Employee/postgres@PostgreSQL 18". The top toolbar includes icons for file operations, query execution, and settings.

Query History

```
-- Query 1
INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');

-- Query 2
INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);

-- Query 3
INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);

-- Query 4
select * from Employees e
left join Salaries s on e.emp_id=s.emp_id
left join Departments d on e.dept_id=d.dept_id;

-- Query 5
update Salaries set salary=salary+salary*0.1 where emp_id
IN (Select emp_id from Employees e join Departments d
on e.dept_id=d.dept_id where dept_name='HR');
```

Data Output

	emp_id [PK] integer ↕	salary integer ↕
1	3	80000
2	4	40000
3	5	45000
4	1	55000

The bottom status bar indicates "Showing rows: 1 to 4" and "Page No: 1 of 1". Navigation controls are visible next to the row count.

Employee/postgres@PostgreSQL 18

Query

Query History

```
1 DROP table Departments, Employees, Salaries;
2 CREATE TABLE Departments (
3     dept_id INT PRIMARY KEY,
4     dept_name VARCHAR(50) NOT NULL
5 );
6
7 CREATE TABLE Employees (
8     emp_id INT PRIMARY KEY,
9     name VARCHAR(50) NOT NULL,
10    dept_id INT REFERENCES Departments(dept_id)
11 );
12
13 CREATE TABLE Salaries (
14     emp_id INT PRIMARY KEY REFERENCES Employees(emp_id) ON DELETE CASCADE,
15     salary INT NOT NULL
16 );
17
18
19 INSERT INTO Departments (dept_id, dept_name) VALUES
20 (10, 'HR'), (20, 'IT'), (30, 'Sales');
21
```

Data Output

Messages

Notifications

emp_id
[PK] integer

Showing rows: 1 to 1

Page No: 1 of 1

1	3
---	---



Experiment 01

(Amazon)

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`.

Every customer must choose **exactly 3 different toppings** to prepare a pizza.

Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.

The same topping **cannot appear more than once** in a pizza.

Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by **highest total cost**

If two pizzas have the same cost, sort them alphabetically.

Employee/postgres@PostgreSQL 18

No limit

Query Query History

```

1 CREATE TABLE pizza_toppings (
2     topping_name VARCHAR(50) PRIMARY KEY,
3     ingredient_cost NUMERIC(4,2) NOT NULL
4 );
5
6
7 INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
8 ('Pepperoni', 0.50),
9 ('Sausage', 0.70),
10 ('Chicken', 0.55),
11 ('Onions', 0.25),
12 ('Extra Cheese', 0.40);
13
14 select CONCAT(T1.topping_name,',',T2.topping_name,',',T3.topping_name) as pizza,
15 (T1.ingredient_cost + T2.ingredient_cost+ T3.ingredient_cost) as total_cost from pizza_toppings T1
16 join pizza_toppings T2
17 on T2.topping_name>T1.topping_name
18 join pizza_toppings T3
19 on T3.topping_name>T2.topping_name
20 Order by total_cost DESC
21

```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	pizza text	total_cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Onions,Extra Cheese,Pepperoni	1.25
10	Pepperoni,Extra Cheese,Pepperoni	1.20

Total rows: 10 Query complete 00:00:05.974

LF Ln 15, Col 1